



Practical DBMS

Prof. Joseph G. Vella ©
Dept. of Computer Information Systems
JOSEPH.G.VELLA@UM.EDU.MT

1



Restart

Revision and Realities

2



The ReSTART provides

- **Explicit presentation of**
 - SQL revision and more
- **Implicit message for integrating**
 - Databases, DBMS and software development life cycle with best of practices

3 PDBMS Restart

3



Caveat lector

The examples and data sets are only meant as to the indication of the SQL construct in focus.
No assumptions should be made on applicability (or suitability) or efficiency or relevance!?

4



SQL's FROM Clause

5


emp

Scott – we still have it!?

empno	ename	job	mgr	hiredate	sal	comm	deptno
integer	character va	character varvin	integer	date	numeric	numeric	integer
7369	SMITH	CLERK	7902	1980-12-17	800		20
7499	ALLEN	SALESMAN	7698	1981-02-20	1600	300	30
7521	WARD	SALESMAN	7698	1981-02-22	1250	500	30
7566	JONES	MANAGER	7839	1981-04-02	2975		20
7654	MARTIN	SALESMAN	7698	1981-09-28	1250	1400	30
7698	BLAKE	MANAGER	7839	1981-05-01	2850		30
7782	CLARK	MANAGER	7839	1981-06-09	2450		10
7788	SCOTT	ANALYST	7566	1982-12-09	3000		20
7844	TURNER	SALESMAN	7698	1981-09-08	1500	0	30
7876	ADAMS	CLERK	7788	1983-01-12	1100		20
7900	JAMES	CLERK	7698	1981-12-03	950		30
7902	FORD	ANALYST	7566	1981-12-03	3000		20
7934	MILLER	CLERK	7782	1982-01-23	1300		10
7839	KING	PRESIDENT		1981-11-17	5000		10

dept

deptno	dname	loc
integer	character varying(14)	character varying(13)
10	ACCOUNTING	NEW YORK
20	RESEARCH	DALLAS
30	SALES	CHICAGO
40	OPERATIONS	BOSTON

salgrade

grade	losal	hisal
integer	integer	integer
1	700	1200
2	1201	1400
3	1401	2000
4	2001	3000
5	3001	9999

6

PDBMS Restart

6



Joins in SQL III – Simple inner (2 tables)

```
-- q1
SELECT e.ename, d.dname
  FROM scott.emp e, scott.dept d
 WHERE e.deptno = d.deptno;

-- q2
SELECT e.ename, d.dname
  FROM scott.emp AS e
 INNER JOIN scott.dept AS d
    ON (e.deptno = d.deptno);
```

ename character varying(10)	dname character varying(14)
SMITH	RESEARCH
ALLEN	SALES
WARD	SALES
JONES	RESEARCH
MARTIN	SALES
BLAKE	SALES
CLARK	ACCOUNTING
SCOTT	RESEARCH
TURNER	SALES
ADAMS	RESEARCH
JAMES	SALES
FORD	RESEARCH
MILLER	ACCOUNTING
KING	ACCOUNTING

No department 'Operations'!

7

PDBMS Restart

7



Joins in SQL III – Simple inner (3 tables)

```
-- q1
SELECT e.ename "Emp", m.ename AS "Super", d.dname AS "Dept."
  FROM scott.emp AS e
 INNER JOIN scott.emp AS m ON (e.mgr = m.empno)
 INNER JOIN scott.dept AS d ON (e.deptno = d.deptno);
```

Emp character varying(10)	Super character varying(10)	Dept. character varying(14)
SMITH	FORD	RESEARCH
ALLEN	BLAKE	SALES
WARD	BLAKE	SALES
JONES	KING	RESEARCH
MARTIN	BLAKE	SALES
BLAKE	KING	SALES
CLARK	KING	ACCOUNTING
SCOTT	JONES	RESEARCH
TURNER	BLAKE	SALES
ADAMS	SCOTT	RESEARCH
JAMES	BLAKE	SALES
FORD	JONES	RESEARCH
MILLER	CLARK	ACCOUNTING

Joins clauses nest left to right.

No employee 'King' or department 'Operations'!

8

PDBMS Restart

8



Joins in SQL III – Outer (2 tables)

```
-- q1
SELECT e.ename, d.dname
  FROM scott.emp AS e
 RIGHT OUTER JOIN scott.dept AS d
    ON (e.deptno = d.deptno);

-- q2 (equivalent to q1 above)
SELECT e.ename, d.dname
  FROM scott.emp e, scott.dept d
 WHERE e.deptno = d.deptno
 UNION
SELECT null, outd.dname
  FROM scott.dept outd
 WHERE outd.deptno NOT IN
    (SELECT DISTINCT deptno
      FROM scott.emp);
```

ename	dname
character varying(10)	character varying(14)
SMITH	RESEARCH
ALLEN	SALES
WARD	SALES
JONES	RESEARCH
MARTIN	SALES
BLAKE	SALES
CLARK	ACCOUNTING
SCOTT	RESEARCH
TURNER	SALES
ADAMS	RESEARCH
JAMES	SALES
FORD	RESEARCH
MILLER	ACCOUNTING
KING	ACCOUNTING
	OPERATIONS

Null!

9 PDBMS Restart

9



Joins in SQL III – Outer (3 tables)

```
-- q1
SELECT e.ename "Emp", m.ename AS "Super", d.dname AS "Dept."
  FROM scott.emp AS e
 LEFT OUTER JOIN scott.emp AS m ON (e.mgr = m.empno)
 RIGHT OUTER JOIN scott.dept AS d ON (e.deptno = d.deptno);
```

Emp	Super	Dept.
character varying(10)	character varying(10)	character varying(14)
SMITH	FORD	RESEARCH
ALLEN	BLAKE	SALES
WARD	BLAKE	SALES
JONES	KING	RESEARCH
MARTIN	BLAKE	SALES
BLAKE	KING	SALES
CLARK	KING	ACCOUNTING
SCOTT	JONES	RESEARCH
TURNER	BLAKE	SALES
ADAMS	SCOTT	RESEARCH
JAMES	BLAKE	SALES
FORD	JONES	RESEARCH
MILLER	CLARK	ACCOUNTING
KING		ACCOUNTING
		OPERATIONS

Null!

10 PDBMS Restart

10



Joins in SQL III – Full outer joins

- NOTE!**
Amend data for 'King' (empno=7839) such that attribute deptno is set to 'null'.

```
-- q1
SELECT e.ename, d.dname
FROM scott.emp AS e
INNER JOIN scott.dept AS d
ON (e.deptno=d.deptno);
```

```
-- q2
SELECT e.ename, d.dname
FROM scott.emp AS e
FULL OUTER JOIN scott.dept AS d
ON (e.deptno=d.deptno);
```

ename	dname
character varying(10)	character varying(14)
SMITH	RESEARCH
ALLEN	SALES
WARD	SALES
JONES	RESEARCH
MARTIN	SALES
BLAKE	SALES
CLARK	ACCOUNTING
SCOTT	RESEARCH
TURNER	SALES
ADAMS	RESEARCH
JAMES	SALES
FORD	RESEARCH
MILLER	ACCOUNTING

No employee 'King' or department 'Operations'!

ename	dname
character varying(10)	character varying(14)
SMITH	RESEARCH
ALLEN	SALES
WARD	SALES
JONES	RESEARCH
MARTIN	SALES
BLAKE	SALES
CLARK	ACCOUNTING
SCOTT	RESEARCH
TURNER	SALES
ADAMS	RESEARCH
JAMES	SALES
FORD	RESEARCH
MILLER	ACCOUNTING
KING	OPERATIONS

11 Null! PDBMS Restart

11



Joins in SQL III – Cartesian product

```
-- q1
SELECT d.dname, s.*
FROM scott.dept d
CROSS JOIN scott.salgrade s
ORDER BY d.dname;
```

dname	grade	losal	hisal
character varying(14)	integer	integer	integer
ACCOUNTING	1	700	1200
ACCOUNTING	2	1201	1400
ACCOUNTING	3	1401	2000
ACCOUNTING	4	2001	3000
ACCOUNTING	5	3001	9999
OPERATIONS	1	700	1200
OPERATIONS	2	1201	1400
OPERATIONS	3	1401	2000
OPERATIONS	4	2001	3000
OPERATIONS	5	3001	9999
RESEARCH	1	700	1200
RESEARCH	2	1201	1400
RESEARCH	3	1401	2000

12 PDBMS Restart

12



Some nifty FROM clauses (i)

```
-- q1 - note the mixing of row with aggregate
SELECT eout.ename, eout.job,
       jobsalscale.minsal,
       eout.sal, jobsalscale.maxsal
FROM (SELECT e.job, min(e.sal), max(e.sal)
      FROM scott.emp e
      GROUP BY e.job)
AS jobsalscale(job,minsal,maxsal)
INNER JOIN scott.emp eout
ON (eout.job = jobsalscale.job);
```

ename	job	minsal	sal	maxsal
character	character vary	numeric	numeric	numeric
SMITH	CLERK	800	800	1300
ALLEN	SALESMAN	1250	1600	1600
WARD	SALESMAN	1250	1250	1600
JONES	MANAGER	2450	2975	2975
MARTIN	SALESMAN	1250	1250	1600
BLAKE	MANAGER	2450	2850	2975
CLARK	MANAGER	2450	2450	2975
SCOTT	ANALYST	3000	3000	3000

```
-- q2 - note the mixing of row with aggregate
SELECT jobsalscale.job, jobsalscale.lower,
       jobsalscale.upper, sg.losal, sg.hisal
FROM (SELECT e.job,
             min(e.sal), max(e.sal),
             avg(e.sal)::int
      FROM scott.emp e
      GROUP BY e.job)
AS jobsalscale(job,lower, upper, average)
INNER JOIN scott.salgrade sg
ON (jobsalscale.average
    BETWEEN sg.losal AND sg.hisal);
```

job	lower	upper	losal	hisal
character varying(9)	numeric	numeric	integer	integer
CLERK	800	1300	700	1200
SALESMAN	1250	1600	1201	1400
MANAGER	2450	2975	2001	3000
ANALYST	3000	3000	2001	3000
PRESIDENT	5000	5000	3001	9999

13PDBMS Restart

13



Some nifty FROM clauses (ii)

```
-- q1
SELECT sgdesc.dsc, sg.losal, sg.hisal
FROM (VALUES
      (1, 'Lowest end'),
      (2, 'Low end'),
      (3, 'Average'),
      (4, 'High end')
     ) as sgdesc(ref,dsc)
INNER JOIN scott.salgrade sg
ON (sg.grade = sgdesc.ref);
```

dsc	losal	hisal
text	integer	integer
Lowest end	700	1200
Low end	1201	1400
Average	1401	2000
High end	2001	3000

14PDBMS Restart

14



Top *n* Queries (iii)

```
-- q1 - top 5
SELECT e.ename, e.sal
FROM scott.emp e
ORDER BY e.sal LIMIT 5;
```

ename	sal
character	numeric
SMITH	800
JAMES	950
ADAMS	1100
MARTIN	1250
WARD	1250

```
-- q2
SELECT e.ename, e.sal
FROM scott.emp e
ORDER BY e.sal LIMIT 3 OFFSET 2;
```

ename	sal
character	numeric
ADAMS	1100
MARTIN	1250
WARD	1250

```
-- q3 - 3 rd (if no duplicates)
SELECT e.ename, e.sal
FROM scott.emp e
ORDER BY e.sal LIMIT 1 OFFSET 2;
```

ename	sal
character	numeric
ADAMS	1100

15

PDBMS Restart

15



Data generating functions/operators (iv)

- The function `generate_series()` is returns a number of rows – aka *table functions*.
 - For example:
- Row returning functions can also be used in the FROM clause.
 - For example:

```
-- q1
SELECT
generate_series(10,50,10)
AS datum;
```

datum
integer
10
20
30
40
50

```
-- q2
SELECT d.i, sg.losal, sg.hisal
FROM generate_series(1,10,1) AS d(i)
LEFT OUTER JOIN scott.salgrade sg
ON (sg.grade = d.i);
```

i	losal	hisal
integer	integer	integer
1	700	1200
2	1201	1400
3	1401	2000
4	2001	3000
5	3001	9999
6		
7		
8		
9		
10		

Nulls!

16

PDBMS Restart

16



Data generating functions/operators (iv) – continued

- Row returning functions can also be used in the FROM clause.
 - For example:
- Another example (to generate dates) is:

```
-- q2
SELECT d.i, sg.losal, sg.hisal
FROM generate_series(1,10,1) AS d(i)
LEFT OUTER JOIN scott.salgrade sg
ON (sg.grade = d.i);
```

i	losal	hisal
integer	integer	integer
1	700	1200
2	1201	1400
3	1401	2000
4	2001	3000
5	3001	9999
6		
7		
8		
9		
10		

Nulls!

```
SELECT
  DATE '2010-01-01'
  + (interval '1' month
    * generate_series(0,5)
    ) AS months;

"2010-01-01 00:00:00"
...
"2010-06-01 00:00:00"
```

17

PDBMS Restart

17



Data generating functions/operators (iv): continued WITH ORDINALITY in FROM Clause

- With functions in the FROM clause one can add the qualifier WITH ORDINALITY to add a new column to the output with a sequence numbering as it values. For example:

```
SELECT *
FROM generate_series(10,100,5)
WITH ORDINALITY AS t(datum,index);

10;1
15;2
...
100;19

SELECT *
FROM generate_series( '2010-01-01 00:00'::timestamp,
  '2010-01-31 23:59'::timestamp,
  '24 hours')
WITH ORDINALITY AS t(datum,numb);

"2010-01-01 00:00:00";1
...
"2010-01-31 00:00:00";31
```

18

PDBMS Restart

18



Data generating functions/operators (iv) - continuation

- Creating functions like `generate_series()` is easy.
 - For example:
- Another example, this time it is written in PLPGSQL.

```
CREATE FUNCTION getEmpsofJob(char)
RETURNS SETOF scott.emp
AS
'SELECT * FROM scott.emp WHERE job=$1;'
LANGUAGE 'sql';

-- q1
SELECT e.ename, e.job, e.mgr
FROM getEmpsofJob('CLERK') AS e;
```

ename	job	mgr
character	character	integer
SMITH	CLERK	7902
ADAMS	CLERK	7788
JAMES	CLERK	7698
MILLER	CLERK	7782

```
CREATE FUNCTION getEmpsofLoc(char)
RETURNS SETOF scott.emp
AS '
DECLARE c_emp scott.emp%ROWTYPE;
BEGIN
  FOR c_emp IN
    SELECT *
      FROM scott.emp AS e
     WHERE e.deptno =
           (SELECT d.deptno
            FROM scott.dept AS d
           WHERE loc=$1)
  LOOP
    RETURN NEXT c_emp;
  END LOOP;
RETURN;
END '
LANGUAGE 'plpgsql';

-- q2
SELECT e.ename, e.job, e.mgr
FROM getEmpsofLoc('DALLAS') AS e;
```

ename	job	mgr
character	character	integer
SMITH	CLERK	7902
JONES	MANAGER	7839
SCOTT	ANALYST	7566
ADAMS	CLERK	7788
FORD	ANALYST	7566

19 PDBMS Restart

19



SQL's Data types - not the usual suspects

20



Aside: type casting in PostgreSQL

- Many times an expression's, or an attribute's, assigned value needs to be encoded as a specific data type.
- Type casting is possible in SQLIII, and in PostgreSQL we use the **CAST()** function - or the '::' operator.
 - The right hand side query ensures that the third output attribute is of type integer.

The expression with the **CAST()** is:

```
SELECT job, avg(sal), CAST(avg(sal) AS integer)
FROM scott.emp
GROUP BY job;
```

job	avg	avg
character varying (9)	numeric	integer
1 CLERK	1037.500000000	1038
2 PRESIDENT	5000.000000000	5000
3 MANAGER	2758.333333333	2758
4 SALESMAN	1400.000000000	1400
5 ANALYST	3000.000000000	3000

21

PDBMS Restart

21



Bit string data type

- Bit strings as a sequence of binary numbers 0 and 1. The bit string PostgreSQL data types are:
 - **bit(n)** – with string length n being fixed at definition;
 - **bit varying(n)** – with string length n being the maximum length supported (e.g. assignable).
- A number of operators are supported by PostgreSQL:
 - Concatenation ||
 - Bitwise AND &
 - Bitwise OR |
 - Bitwise #
 - Bitwise ~
 - Bitwise shift
 - Left <<
 - Right >>

l	r	r bitAND l	r bitOR l
bit	bit	bit	bit
1 00001101	00000100	00000100	00001101

22

PDBMS Restart

22



Sequence aka Serial

Useful and handy - **but** do not subscribe (i.e. imply) order or continuation!

```
-- DDL for sequence
CREATE SEQUENCE scott.depend_serno;

-- DDL for table with a sequence
CREATE TABLE scott.depend(
  serno INTEGER PRIMARY KEY
    DEFAULT nextval('scott.depend_serno'),
  dname CHARACTER VARYING(10) NOT NULL,
  empno INTEGER);

-- attach sequence to table.attr
-- if exclusivity required
ALTER SEQUENCE scott.depend_serno
  OWNED BY scott.depend.serno;

-- DML
INSERT INTO scott.depend(dname,empno)
VALUES ('Joe',1122),
       ('Jay',2233),
       ('Jon',3344),
       ('Jil',4455);
```

serno	dname	empno
integer	character	integer
1	Joe	1122
2	Jay	2233
3	Jon	3344
4	Jil	4455

Generated from sequence.

23 PDBMS Restart

23



Serial - continued

```
-- DML
INSERT INTO
  scott.depend(serno,dname,empno)
VALUES (1111,'Ono',9911);
```

This Works?

```
-- serial object related functions

-- sets next value and returns (can be wasted)
SELECT nextval('scott.depend_serno');

-- set counter to 100 - nextval will return 101
SELECT setval('scott.depend_serno', 100);

-- set counter to 100 - nextval will return *100*
SELECT setval('scott.depend_serno', 100, false);
```

24 PDBMS Restart

24



ENUMerated types

Defining our own types – we start with a very simple example.

```
-- ddl
CREATE TYPE scott.rainbow
AS ENUM ('red','orange','yellow','green',
'blue','indigo','violet');
-- note case sensitive and order is important

CREATE TABLE scott.likecolour(
empno    INTEGER PRIMARY KEY,
likecol  scott.rainbow NOT NULL);

-- dml
INSERT INTO scott.likecolour
VALUES
(1122,'red'),
(2233,'violet'),
(3344,'blue'),
(4455,'blue');
```

25



ENUMerated types: continued

```
-- q1 -- data dictionary view
-- explains the order of the
-- enumerated type entries
SELECT *
FROM pg_enum;
```

enumtypid	enums	enumlabel
oid	real	name
24602	1	red
24602	2	orange
24602	3	yellow
24602	4	green
24602	5	blue
24602	6	indigo
24602	7	violet

```
-- q2
SELECT *
FROM scott.likecolour AS lc
WHERE likecol > 'green';
```

empno	likecol
integer	scott.rainbow
2233	violet
3344	blue
4455	blue

```
-- q3 -- note the order
SELECT *
FROM scott.likecolour AS lc
ORDER BY likecol;
```

empno	likecol
integer	scott.rainbow
1122	red
3344	blue
4455	blue
2233	violet

26



BLOB – basic binary data type - ByteA

- ByteA support binary strings in bytes.
- The "TOAST" (The Oversized-Attribute Storage Technique) storage system, which is supported by PostgreSQL, automatically stores values bigger than a single database page (usually 8 KB) into a secondary storage region for each table. This limits the size of any field or column to 1 GB.
- PostgreSQL is close to *SQL/III* standard binary string type 'BLOB'.
- Examples of binary blobs:
 - JPEGs, TIFF
 - WAV, MP3
 - Executable code



Reading directly from the server process is a security risk!

- The following scripts assume database user is a "superuser".

27

PDBMS Restart

27



BLOB – basic binary data type – ByteA (continued)

Import this image (on the right).

- Create table:


```
CREATE TABLE raster.image
(i_sn  serial primary key,
 i_img bytea);
```
- Create a function to read image (into table):


```
CREATE OR REPLACE FUNCTION
  raster.bytea_import
    (p_path text,
     p_result out bytea)
LANGUAGE plpgsql AS $$
-- https://dba.stackexchange.com/questions
-- /1742/how-to-insert-file-data-into
-- -a-postgresql-bytea-column. (Jack Douglas, 2011)
DECLARE
  l_oid oid;
BEGIN
  select lo_import(p_path) into l_oid;
  select lo_get(l_oid) into p_result;
  perform lo_unlink(l_oid);
END;$$;
```



- Read image into an attribute in the table:


```
INSERT INTO raster.image(i_img)
  SELECT raster.bytea_import(
    'base/jpgs/Slidel.jpeg' );
```
- Check the size of the read image (and compare to the original).


```
SELECT LENGTH(i_img) AS filesize
  FROM raster.image;
-- filesize(integer)
-- 68166
```

28

PDBMS Restart

28



SQL's Data types: The ARRAY[] type constructor

29



SQL and Arrays

- Arrays are popular data structure used in GUI elements, computations and even visualisations.
- We are going to provide a quick overview how array type constructor can we used to type a table's attribute.
 - Plus, the support SQL needs to define, update, manage and query tables with arrays.

```
-- q1
SELECT ARRAY[1::INT,2,3,4,5,6];
```

array
integer[]
{1,2,3,4,5,6}

```
-- q2
SELECT ARRAY[ARRAY[1::int,101::int],
              ARRAY[2,201],
              ARRAY[3,301],
              ARRAY[4,401],
              ARRAY[5,501],
              ARRAY[6,601]];
```

array
integer[]
{ {1,101}, {2,201}, {3,301}, {4,401}, {5,501}, {6,601} }

```
-- q3
INSERT INTO schema.table (a1, array_a2)
VALUES( 123, '{1,2,3,4,5,6}' );
```

```
-- q4
SELECT ARRAY(SELECT deptno::INT
              FROM scott.dept);
```

array
integer[]
{10,20,30,40}

PDBMS Restart

30



DDL with Arrays

```
-- q1
ALTER TABLE scott.salgrade
  ADD COLUMN emps INT[];

-- q2
-- or bounded array
-- (from 1 to 10 elements)
ALTER TABLE scott.salgrade
  ADD COLUMN emps INT[10];

-- ddl1
CREATE TABLE scott.salgrade
(
  grade integer NOT NULL,
  losal integer,
  hisal integer,
  emps integer[],
  CONSTRAINT salgrade_pkey
  PRIMARY KEY (grade)
);

-- q3
INSERT INTO scott.salgrade
(grade, losal, hisal, emps)
VALUES( 9, 9900, 9999, '{1,2,3,4,5,6}' );
```

31

PDBMS Restart

31



SQL Arrays: Addressing elements

First element is 1 (not 0)

```
-- q1
-- return an item
SELECT (ARRAY[1,2,3,4,5,6])[1]; .....> 1

-- q2
-- return an (sub)array
SELECT (ARRAY[1,2,3,4,5,6])[3:5]; .....> {3,4,5}
SELECT (ARRAY[1,2,3,4,5,6])[1:1]; .....> {1}

-- q3
-- out of bounds returns
-- single element return *null*
SELECT (ARRAY[1,2,3,4,5,6])[10]; .....> ∅

-- q4
-- array returns what's available
SELECT (ARRAY[1,2,3,4,5,6])[3:10]; .....> {3,4,5,6}
-- even an array with no elements
SELECT (ARRAY[1,2,3,4,5,6])[9:10]; .....> {}

-- q5
-- extraction can be repeated
SELECT ((ARRAY[1,2,3,4,5,6])[3:5])[2]; .....> 4
```

32

PDBMS Restart

32



SQL Arrays – Insert and Update

```
-- q1
INSERT INTO scott.salgrade
VALUES (10,20000,25000,
       ARRAY(SELECT deptno
              FROM scott.dept));

SELECT *
FROM scott.salgrade
WHERE grade = 10;
```

grade	losal	hisal	emps
integer	integer	integer	integer[]
10	20000	25000	{10,20,30,40}

```
-- single array entry
UPDATE scott.salgrade
SET emps[5]=50
WHERE grade=10;
```

grade	losal	hisal	emps
integer	integer	integer	integer[]
10	20000	25000	{10,20,30,40,50}

```
-- range array entry
UPDATE scott.salgrade
SET emps[6:9]=ARRAY[60,70,80,90]
WHERE grade=10;
```

grade	losal	hisal	emps
integer	integer	integer	integer[]
10	20000	25000	{10,20,30,40,50,60,70,80,90}

33

PDBMS Restart

33



SQL: arrays to tuples

```
-- q1
SELECT unnest(ARRAY[1,2,3,4,5,6]);

-- q2
SELECT unnest(
  ARRAY[ARRAY[1,101],
        ARRAY[2,201],
        ARRAY[3,301],
        ARRAY[4,401],
        ARRAY[5,501],
        ARRAY[6,601]]);

-- q3
SELECT *
FROM unnest(ARRAY[1,2,3,4,5,6],
            ARRAY['a','b','c','d'],
            ARRAY[100,200])
AS t(a,b,c);
```

unnest	integer
1	1
2	2
3	3
4	4
5	5
6	6

unnest	integer
1	101
2	201
3	301
4	401
5	501
6	601

a	b	c
integer	text	integer
1	a	100
2	b	200
3	c	
4	d	
5		
6		

34

PDBMS Restart

34



SQL: tuples to arrays

```
-- q1
SELECT sg.grade, count(*)
  FROM scott.emp e
 INNER JOIN scott.salgrade sg
    ON (sg.losal <= e.sal AND e.sal <= sg.hisal)
 GROUP BY sg.grade
 ORDER BY sg.grade;
```

grade	count
integer	bigint
1	3
2	3
3	2
4	5
5	1

```
-- q2
SELECT sg.grade, count(*), array_agg(e.empno)
  FROM scott.emp e
 INNER JOIN scott.salgrade sg
    ON (sg.losal <= e.sal AND e.sal <= sg.hisal)
 GROUP BY sg.grade
 ORDER BY sg.grade;
```

grade	count	array_agg
integer	bigint	integer[]
1	3	{7369,7876,7900}
2	3	{7521,7654,7934}
3	2	{7499,7844}
4	5	{7566,7698,7782,7788,7902}
5	1	{7839}

```
-- this should be a view defn. really
UPDATE scott.salgrade sg
  SET emps =(SELECT array_agg(empno)
              FROM scott.emp e
              WHERE sg.losal <= e.sal
                AND e.sal <= sg.hisal)
 WHERE sg.grade IN (1,2,3,4,5);
```

grade	losal	hisal	emps
integer	integer	integer	integer[]
1	700	1200	{7369,7876,7900}
2	1201	1400	{7521,7654,7934}
3	1401	2000	{7499,7844}
4	2001	3000	{7566,7698,7782,7788,7902}
5	3001	9999	{7839}

35

PDBMS Restart

35



SQL: Selecting with arrays

```
-- q1
SELECT *
  FROM scott.salgrade sg;
```

grade	losal	hisal	emps
integer	integer	integer	integer[]
1	700	1200	{7369,7876,7900}
2	1201	1400	{7521,7654,7934}
3	1401	2000	{7499,7844}
4	2001	3000	{7566,7698,7782,7788,7902}
5	3001	9999	{7839}

```
-- q2
SELECT *
  FROM scott.salgrade sg
 WHERE sg.emps[1] = 7499;
```

grade	losal	hisal	emps
integer	integer	integer	integer[]
3	1401	2000	{7499,7844}

```
-- q3
SELECT *
  FROM scott.salgrade sg
 WHERE 7900 = ANY(emps);
```

grade	losal	hisal	emps
integer	integer	integer	integer[]
1	700	1200	{7369,7876,7900}

```
-- q4
SELECT *
  FROM scott.salgrade sg
 WHERE 7839 = ALL(emps);
```

grade	losal	hisal	emps
integer	integer	integer	integer[]
5	3001	9999	{7839}

```
-- q5
SELECT *
  FROM scott.salgrade sg
 WHERE 7500 <= ALL(emps);
```

grade	losal	hisal	emps
integer	integer	integer	integer[]
2	1201	1400	{7521,7654,7934}
4	2001	3000	{7566,7698,7782,7788,7902}
5	3001	9999	{7839}

36

PDBMS Restart

36



SQL's Array operators

```
-- q1 - equals =
SELECT ARRAY[1,2,3] = ARRAY[1,2,3] AS "Equal";
SELECT ARRAY[1,2,3] = ARRAY[3,2,1] AS "Equal";

-- q2 - contains @> and is contained <@
SELECT ARRAY[1,2,3] @> ARRAY[1,3] AS "Contains";

SELECT ARRAY[1,2,3] <@ ARRAY[1,2,3,4] AS "Contained";

-- q3 - concatenate array + array and element + array
SELECT ARRAY[1,2,3] || ARRAY[4,5,6] AS "Concat";

SELECT 0 || ARRAY[1,2,3] AS "Concat";

-- q4 - && overlap
SELECT ARRAY[1,2,3] && ARRAY[-1,2,4] AS "Overlap";
```

True

False

True

True

Concat
integer[]
{1, 2, 3, 4, 5, 6}

Concat
integer[]
{0, 1, 2, 3}

True

37

PDBMS Restart

37



SQL Array's Functions (i)

```
-- q1s - return the dims
SELECT array_ndims (ARRAY[1,2,3,4,5,6]);

SELECT array_ndims (ARRAY
    [ARRAY[1,101],ARRAY[2,201],ARRAY[3,301],
    ARRAY[4,401],ARRAY[5,501],ARRAY[6,601]]);

SELECT array_dims (ARRAY
    [ARRAY[1,101],ARRAY[2,201],ARRAY[3,301],
    ARRAY[4,401],ARRAY[5,501],ARRAY[6,601]]);

-- q2 - array_length
SELECT array_length (ARRAY[1,2,3,4,5,6], 1);

-- q3 - cardinality
SELECT cardinality (ARRAY[1,2,3,4,5,6]);

SELECT cardinality (ARRAY
    [ARRAY[1,101],ARRAY[2,201],ARRAY[3,301],
    ARRAY[4,401],ARRAY[5,501],ARRAY[6,601]]);
```

1

2

[1:6][1:2]

6

6

12

38

PDBMS Restart

38



SQL Array's Functions (ii)

<pre>-- q1s - array_fill SELECT array_fill(1,ARRAY[3]); SELECT `(1,ARRAY[2,3]);</pre>	<pre>{1,1,1} {{1,1,1},{1,1,1}}</pre>
<pre>-- q2 - array_position SELECT array_position(ARRAY[1,2,3,4,5,6],3);</pre>	<pre>3</pre>
<pre>-- q3 - array_replace SELECT array_replace(ARRAY[1,2,3,4,5,6],3,30);</pre>	<pre>{1,2,30,4,5,6}</pre>
<pre>-- q4s - array_to_string and -- string_to_array SELECT array_to_string(ARRAY['Richard','of','York', 'gave', 'etc'], ' ');</pre>	<pre>"Richard of York gave etc"</pre>
<pre>SELECT string_to_array('asd fghj kl zxcvb nm',' ');</pre>	<pre>{asd,fghj,kl,zxcvb,nm}</pre>

39

PDBMS Restart

39



SQL's Data types: The RANGE type constructor

40



SQL and Ranges

- These are data types representing a spread of values by listing the two endpoints and their bounds (i.e. inclusive or exclusive).
- PostgreSQL support a number of ranges (think as a range of [data type]). The system provides:
 - Integers (and bigint), numeric, date, timestamp (and with timezone);
 - And also User Defined Types (i.e. use CREATE TYPE) - IP addresses.
- Given a range instance in integers, e.g. salary range during an employment, the lower bound is the lowest and the upper bound is the highest.
 - In this case both bounds are inclusive (denoted by '[' and ']') and mean >= and <=.
 - So a typical example is "[1500,2250]".
- Given a range instance in integers, e.g. current salary scale for managers is over 2000 up to and including 3000. The exclusive lower bound is represented by '(''.
 - In this case we have "(2000,3000)".
 - We therefore can mix inclusive and exclusive.
 - We can also have 'empty' bounds too!
 - A bound can be assigned 'infinity' - check semantics.
- Usefull in data and query modelling of:
 - Schedules for job arrivals;
 - Run predicates to determine type of intersection;
 - Intersecting data streams;
 - Coalescing of data / events.
- The above problems are typically hard to code in SQL using basic data types!

41 PDBMS Restart

41



Range encoding

```
-- range encoding
-- note the normalisation.
-- e.g. 9) instead of 8]

SELECT '[2,8]':int4range;
-- [2,9)::int4range

SELECT '[2,8]':int4range;
-- [2,8)::int4range

SELECT '(2,8)':int4range;
-- [3,8)::int4range

SELECT '[4,4]':int4range;
-- empty

-- range creating function
SELECT numrange(1,12);
-- [1,12)::numrange

SELECT daterange
('2010-01-01', '2020-12-31');
-- [2010-01-01,
-- 2020-12-31)::daterange
```

```
-- multirange encoding

SELECT '{[18,20), [25,65]] '
::int4multirange;
-- {[18,20), [25,66]]::int4multirange

SELECT '{[16,18), [18,65]] '
::int4multirange;
-- {[16,66]]::int4multirange
-- note the coalescing of two ranges
-- into a single range

-- convert from multi to range
SELECT unnest('{[18,20), [25,65]] '
::int4multirange);
-- [18,20)::int4range
-- [25,66)::int4range

SELECT unnest('{[16,18), [18,65]] '
::int4multirange);
-- [16,66)::int4range
```

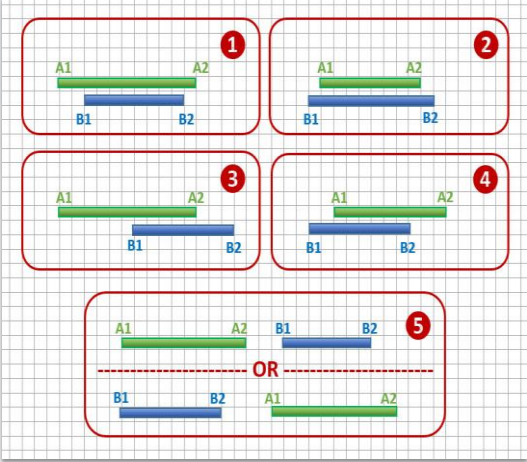
42 PDBMS Restart

42



Range predicates: Contained, Contains, Follow/Precede

- Contained relationship**
 - Fig. 1
Bs contained in As -> TRUE
As contained in Bs -> FALSE
 - Fig. 3
Bs contained in As -> FALSE
- Contains relationship**
 - Fig. 2
Bs contains As -> TRUE
As contains Bs -> FALSE
 - Fig. 4
Bs contains As -> FALSE
- Follows relationship**
 - Fig. 5 (upper)
As follows Bs -> TRUE
Bs follows As -> FALSE



43

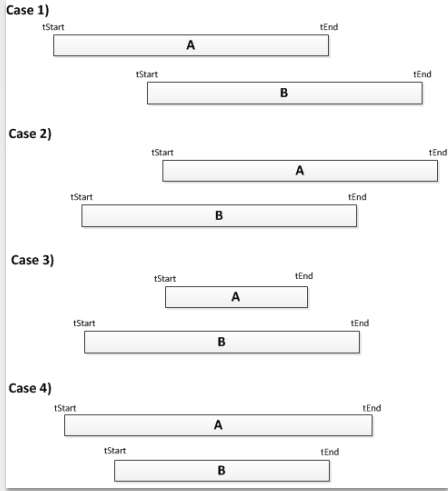
PDBMS Restart

43



Range predicates: Overlap

- Overlap of ranges is true when every range share a common interval with the other.**
- Overlap, e.g. dates, may occur in a number of ways.**
 - The figure on the right gives a diagramtic representation of possible overlap of two date ranges.



44

PDBMS Restart

44



Range predicates: Overlap example

- Let create some ranges from our Scott database – say salary ranges per job title.
The query to realise this (and somemore details) follows:

```
CREATE OR REPLACE VIEW scott.job_sal_range AS
SELECT job,
       min(sal) "minsal", .....
       max(sal) "maxsal",
       numrange(min(sal),
               max(sal),
               '[' ]') "rangesal"
FROM scott.emp
GROUP BY job
ORDER BY minsal ASC;
SELECT * FROM scott.job_sal_range;
```

job	character varying (9)	minsal	numeric	maxsal	numeric	rangesal	numrange
1	CLERK	800		1300		[800,1300]	
2	SALESMAN	1250		1600		[1250,1600]	
3	MANAGER	2450		2975		[2450,2975]	
4	ANALYST	3000		3000		[3000,3000]	
5	PRESIDENT	5000		5000		[5000,5000]	

```
-- overlap query
SELECT l.job || ' and ' || r.job || ' overlap! '
FROM scott.job_sal_range l
INNER JOIN scott.job_sal_range r
ON (l.rangesal && r.rangesal
    AND l.job<r.job );
```

Overlap?	text
1	CLERK and SALESMAN overlap!

45

PDBMS Restart

45



Range Operations: Contains

- Create a view (as in previous) but for department.

```
CREATE OR REPLACE VIEW scott.deptno_sal_range AS
SELECT dept.deptno,
       min(sal) "minsal", .....
       max(sal) "maxsal",
       numrange(min(sal),max(sal), '[' ]')
       "rangesal"
FROM scott.emp
RIGHT OUTER JOIN scott.dept
ON (emp.deptno=dept.deptno)
GROUP BY dept.deptno
ORDER BY minsal ASC;
```

deptno	[PK] integer	minsal	numeric	maxsal	numeric	rangesal	numrange
20		800		3000		[800,3000]	
2		30		950		[950,2850]	
3		10		1300		[1300,5000]	
4		40		[null]		[null]	(.)

- Excute the followng query:

```
SELECT l.rangesal , r.rangesal,
       l.deptno || ' is contained in ' || r.deptno || '!' "Contained?"
FROM scott.deptno_sal_range l
INNER JOIN scott.deptno_sal_range r
ON (l.rangesal <@ r.rangesal AND l.deptno <> r.deptno)
WHERE NOT l.rangesal = '(,)'
AND NOT r.rangesal = '(,)' ;
```

rangesal	numrange	rangesal	numrange	Contained?	text
1	[950,2850]	[800,3000]		30 is contained in 20!	

46

PDBMS Restart

46



Range Operations: Strictly before

- We now write a query to check if a range preceeds another (and with out overlap)
 - strictly before!

job	character varying (9)	minsal	numeric	maxsal	numeric	rangesal	numrange
1	CLERK	800		1300		[800,1300]	
2	SALESMAN	1250		1600		[1250,1600]	
3	MANAGER	2450		2975		[2450,2975]	
4	ANALYST	3000		3000		[3000,3000]	
5	PRESIDENT	5000		5000		[5000,5000]	

```
SELECT l.rangesal , r.rangesal,
       l.job || ' is before in ' || r.job || '!' 'Strictly Left?'
FROM scott.job_sal_range l
INNER JOIN scott.job_sal_range r
      ON (l.rangesal << r.rangesal
          AND l.job <> r.job)
WHERE NOT l.rangesal = '(,)'
      AND NOT r.rangesal = '(,)';
```

	rangesal	numrange	rangesal	numrange	Strictly Left?	text
1	[800,1300]		[2450,2975]			CLERK is before in MANAGER!
2	[800,1300]		[3000,3000]			CLERK is before in ANALYST!
3	[800,1300]		[5000,5000]			CLERK is before in PRESIDENT!
4	[1250,1600]		[2450,2975]			SALESMAN is before in MANAGER!
5	[1250,1600]		[3000,3000]			SALESMAN is before in ANALYST!
6	[1250,1600]		[5000,5000]			SALESMAN is before in PRESIDENT!
7	[2450,2975]		[3000,3000]			MANAGER is before in ANALYST!
8	[2450,2975]		[5000,5000]			MANAGER is before in PRESIDENT!
9	[3000,3000]		[5000,5000]			ANALYST is before in PRESIDENT!

47

PDBMS Restart

47



Range Operations: Manipulation

- How about computing the overlap?
 - Intersect and union example follow.

job	character varying (9)	minsal	numeric	maxsal	numeric	rangesal	numrange
1	CLERK	800		1300		[800,1300]	
2	SALESMAN	1250		1600		[1250,1600]	
3	MANAGER	2450		2975		[2450,2975]	
4	ANALYST	3000		3000		[3000,3000]	
5	PRESIDENT	5000		5000		[5000,5000]	

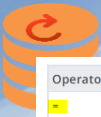
```
SELECT l.job || ' and ' || r.job || ' overlap! ' "Overlap?",
       l.rangesal * r.rangesal "Common",
       l.rangesal + r.rangesal "Total"
FROM scott.job_sal_range l
INNER JOIN scott.job_sal_range r
      ON (l.rangesal && r.rangesal
          AND l.job<r.job );
```

	Overlap?	text	Common	numrange	Total	numrange
1		CLERK and SALESMAN overlap!		[1250,1300]		[800,1600]

48

PDBMS Restart

48



PostgreSQL range operators

Operator	Description	Example	Result
=	equal	int4range(1,5) = '[1,4]':int4range	t
<>	not equal	numrange(1.1,2.2) <> numrange(1.1,2.3)	t
<	less than	int4range(1,10) < int4range(2,3)	t
>	greater than	int4range(1,10) > int4range(1,5)	t
<=	less than or equal	numrange(1.1,2.2) <= numrange(1.1,2.2)	t
>=	greater than or equal	numrange(1.1,2.2) >= numrange(1.1,2.0)	t
@>	contains range	int4range(2,4) @> int4range(2,3)	t
@>	contains element	'[2011-01-01,2011-03-01]':tsrange @> '2011-01-10':timestamp	t
<@	range is contained by	int4range(2,4) <@ int4range(1,7)	t
<@	element is contained by	42 <@ int4range(1,7)	f
&&	overlap (have points in common)	int8range(3,7) && int8range(4,12)	t
<<	strictly left of	int8range(1,10) << int8range(100,110)	t
>>	strictly right of	int8range(50,60) >> int8range(20,30)	t
<	does not extend to the right of	int8range(1,20) < int8range(18,20)	t
>	does not extend to the left of	int8range(7,20) > int8range(5,10)	t
- -	is adjacent to	numrange(1.1,2.2) - - numrange(2.2,3.3)	t
+	union	numrange(5,15) + numrange(10,20)	[5,20]
*	intersection	int8range(5,15) * int8range(10,20)	[10,15]
-	difference	int8range(5,15) - int8range(10,20)	[5,10]

Better definitions are on: <https://www.postgresql.org/docs/current/functions-range.html>

49

PDBMS Restart



Reading and Writing Data

JavaScript Object Notation – *abbreviated as JSON*



JSON

- JSON is a data-interchange format. From:
 - DBMS to DBMS;
 - DBMS to a client process (e.g. common browser objects).
 - Many APIs read instructions and return data in JSONs.
(look at <http://www.sitepoint.com/10-example-json-files/>)
- JSON is language independent.
 - BSON (JSONB in code) is the binary version of JSON.
- JSON data building syntax is easy to understand and apply.
 - It's also easy to read with automated programs.
- **JSON is a of semi-structured datatype – any two docs can have similar but not necessarily must have identical structure.**
- JSON is built on objects which are a composition of key and value pairs. Values can be any instances of the following generics:
 - A number (integer or floating point);
 - A string (in double quotes);
 - A Boolean (true or false);
 - An array (in square brackets);
 - An object (in curly braces);
 - Null.
- **PostgreSQL comes with extensive range of functions to build, tear-apart , verify and search JSONs.**

PDBMS Restart

51

51



JSON examples

```
-- example 1
{"firstName":"John", "lastName":"Doe"}

-- example 2 with an array
{"firstName":"John", "lastName":"Doe",
 "drives" : ["FIAT 126","JAGUAR"]}
```

```
-- example 3 with an array of embedded objects
{"firstName":"John", "lastName":"Doe",
 "manages": [
  {"firstName":"JoJo", "lastName":"Doe"},
  {"firstName":"Anna", "lastName":"Smith"},
  {"firstName":"Peter", "lastName":"Jones"}
]}
```

```
-- JSON validation
-- simple example for smaller files

SELECT '{"firstName":"John", "lastName":"Doe"}'::JSON;

-- problem with array delimiters (ie missing [ & ] )
SELECT '{"firstName":"John", "lastName":"Doe",
 "drives" : "FIAT 126","JAGUAR" }'::JSON;
ERROR:  invalid input syntax for type json
```

PDBMS Restart

52

52



Generate JSON from tables (and write them out – THIS DEPENDS ON O/S RIGHTS)



```
-- q1
copy(
  SELECT row_to_json(r)
    FROM ( SELECT *
            FROM scott.dept) r)
to 'fullyqualifiedfilename.json';
```

deptno	dname	loc
10	ACCOUNTING	NEW YORK
20	RESEARCH	DALLAS
30	SALES	CHICAGO
40	OPERATIONS	BOSTON

```
-- q2
copy(
  SELECT row_to_json(r)
    FROM ( SELECT
            d.deptno,d.dname,d.loc,
            array_agg(e.ename)
              FROM scott.dept d,
              scott.emp e
            WHERE d.deptno=e.deptno
            GROUP BY d.deptno,
                    d.dname,
                    d.loc ) r)
to 'fullyqualifiedfilename.json';
```

deptno	dname	loc	array agg
10	ACCOUNTING	NEW YORK	["CLARK", "MILLER", "KING"]
20	RESEARCH	DALLAS	["SMITH", "JONES", "SCOTT", "ADAMS", "FORD"]
30	SALES	CHICAGO	["ALLEN", "WARD", "MARTIN", "BLAKE", "TURNER", "JAMES"]

53PDBMS Restart

53



JSON functions & operators

```
-- q1
SELECT
  '{"firstName":"John",
   "lastName":"Doe"}'::JSON
->
  'lastName';

→ outputs "Doe"
```

```
-- q2
SELECT
  '{"firstName":"John",
   "lastName":"Doe"}'::JSON
->>
  'lastName';

→ outputs Doe as text datatype
```

```
-- q3 path expressions
SELECT
  '{"firstName":"John",
   "lastName":"Doe",
   "drives" : ["FIAT 126","JAGUAR"]}'
  ::JSON
  #> '{drives,1}';
-- outputs "JAGUAR"
```

54PDBMS Restart

54



JSON functions & operators (continued)

```
-- q1
-- JSON containment (at top level)
-- Note JSONB data type and not JSON
SELECT
  '{"firstName":"John",
    "drives" : ["FIAT 126","JAGUAR"]}'
  ::JSONB
@> '{"firstName":"John"}'::JSONB;
→ outputs true

-- q2
-- key exists?
SELECT
  '{"firstName":"John",
    "lastName":"Doe"}'::JSONB
? 'lastName';
→ outputs true

-- q3
-- json concat
SELECT
  '{"firstName":"John",
    "lastName":"Doe"}\'
  ::JSONB
||
  '{"middleName":"Peter-Paul"}\'
  ::JSONB;
→ outputs '{"lastName": "Doe",
"firstName": "John", "middleName":
"Peter-Paul"}'

-- q4
-- json delete key/value pair
SELECT
  '{"firstName":"John",
    "lastName":"Doe"}\'
  ::JSONB
-
  'lastName';
→ outputs '{"firstName": "John"}'
```

55

PDBMS Restart

55



JSON operators: check for JSONB

Operator	Right Operand Type	Description	Example	Example Result
->	int	Get JSON array element (indexed from zero, negative integers count from the end)	'[{"a": "foo"}, {"b": "bar"}]'::json->2	{"c": "baz"}
->	text	Get JSON object field by key	'{"a": {"b": "foo"}}'::json->'a'	{"b": "foo"}
->>	int	Get JSON array element as text	'[1,2,3]'::json->>2	3
->>	text	Get JSON object field as text	'{"a":1,"b":2}'::json->>'b'	2
#>	text[]	Get JSON object at specified path	'{"a": {"b": {"c": "foo"}}}'::json#>'a,b'	{"c": "foo"}
#>>	text[]	Get JSON object at specified path as text	'{"a": [1,2,3], "b": [4,5,6]}\'::json#>>'a,2'	3

Better definitions are on: <https://www.postgresql.org/docs/current/functions-json.html>

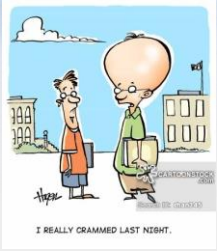
56

PDBMS Restart

56



Revise & Restart!?



- **Intro to DB:**
 - ERMs;
 - RDB and SQL
 - Including Rational Algebra;**
 - Stored Procedures
 - **in PostgreSQL we have both stored functions & procedures.**
- **Ensure a recent version of PostgreSQL is installed at you.**
 - Revise: Back-up a database and then try to import it into new/separate database instance.

57

PDBMS Restart

57